Clasificando Fashion-MNIST con PyTorch

Victoria Lopez*
Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Ciudad Universitaria, Córdoba, Argentina
(Dated: November 18, 2024)

En este trabajo se presenta la implementación de un modelo de red neuronal utilizando PyTorch para clasificar imágenes del conjunto de datos Fashion-MNIST, utilizando un dataset en tareas de aprendizaje automático para la clasificación de prendas de vestir.

I. INTRODUCCIÓN

En este informe se aborda la implementación y simulación de un modelo de red neuronal utilizando el framework PyTorch, el objetivo principal es entrenar un modelo capaz de clasificar imágenes del conjunto de datos Fashion-MNIST, el cual contiene imágenes en escala de grises de prendas de vestir, organizadas en 10 categorías.

El modelo propuesto tiene varias capas, incluyendo dos capas ocultas que utilizan funciones de activación ReLU, y aplica dropout para reducir el riesgo de sobreajuste. Además, durante el entrenamiento, se utilizan métodos de optimización como Stochastic Gradient Descent (SGD) o ADAM, y se mide el rendimiento del modelo con la función de pérdida de entropía cruzada (Cross Entropy Loss).

A lo largo del informe, se presentan simulaciones computacionales y métricas que exploran el impacto de distintos hiperparámetros, como el tamaño del lote, la tasa de aprendizaje y la estructura de la red. Dichos resultados nos permiten comprender mejor el comportamiento de las redes neuronales.

II. TEORÍA

En este proyecto, se utiliza una red neuronal multicapa para clasificar imágenes del conjunto de datos Fashion-MNIST. Este dataset contiene imágenes en escala de grises de 28x28 píxeles, que representan prendas y accesorios organizados en 10 categorías. Cada imagen es procesada como un conjunto de valores numéricos normalizados, que son la entrada para el modelo.

El modelo se compone de las siguientes capas:

- **Capa de entrada:** Esta capa toma los valores de cada píxel de la imagen (784 en total) y los organiza en un formato adecuado para ser procesados por la red.
- **Capas ocultas:** El modelo incluye dos capas ocultas, diseñadas con 128 y 64 neuronas, respectivamente. Estas capas utilizan funciones de activación **ReLU**, que permiten obtener relaciones no lineales en los datos. Además, utilizamos el **dropout**, que aleatoriamente desactiva algunas neuronas durante el entrenamiento para prevenir el sobreajuste.
- **Capa de salida:** La última capa contiene 10 neuronas, donde cada una representa una categoría del conjunto de datos. La red produce una probabilidad para cada categoría, indicando a cuál de ellas pertenece la imagen.

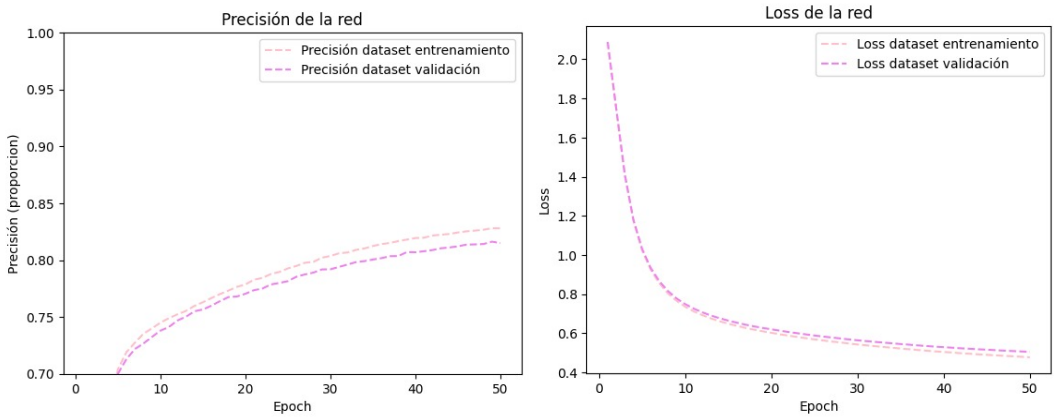
El entrenamiento del modelo se realiza mediante un proceso iterativo donde se ajustan los pesos de las conexiones entre neuronas para minimizar el error en las predicciones. Este ajuste se basa en el cálculo de un gradiente que mide cómo cambia el error respecto a cada peso. Para esto, utilizamos el método de optimización de ADAM o Stochastic Gradient Descent (SGD), que son más eficientes.

Para evaluar el desempeño del modelo, utilizamos Cross Entropy Loss, que mide la diferencia entre las probabilidades predichas y las etiquetas reales de las imágenes, y la precisión del modelo, que calculamos como la proporción de imágenes correctamente clasificadas en los conjuntos de entrenamiento y validación.

Realizamos simulaciones donde se exploran diferentes configuraciones del modelo, como el tamaño de los lotes, la tasa de aprendizaje y el nivel de dropout, lo que nos permite analizar cómo estos parámetros influyen en la capacidad del modelo para aprender.

III. RESULTADOS

A. Modelo inicial



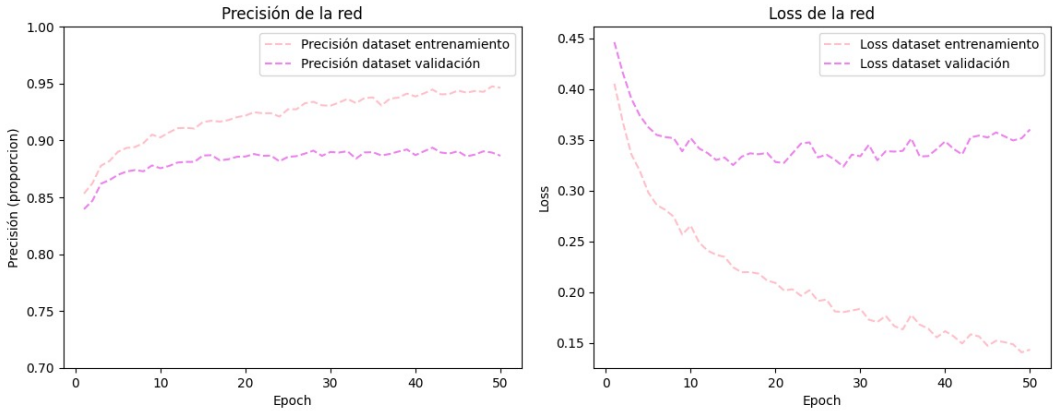
(a) Precisión del modelo inicial en entrenamiento y testeo

(b) Loss del modelo inicial en entrenamiento y testeo

Figure 1: Resultados del modelo inicial

En estos gráficos podemos notar cómo, al inicio del entrenamiento, la pérdida disminuye rápidamente y luego se estabiliza en valores bajos, mientras que la precisión aumenta y se estabiliza, con el conjunto de validación ligeramente por debajo del de entrenamiento. Esto nos muestra que el modelo generaliza de forma correcta y nos permite ver que el aprendizaje podría mejorar con más ajustes o más épocas.

B. Modelo con optimizador ADAM



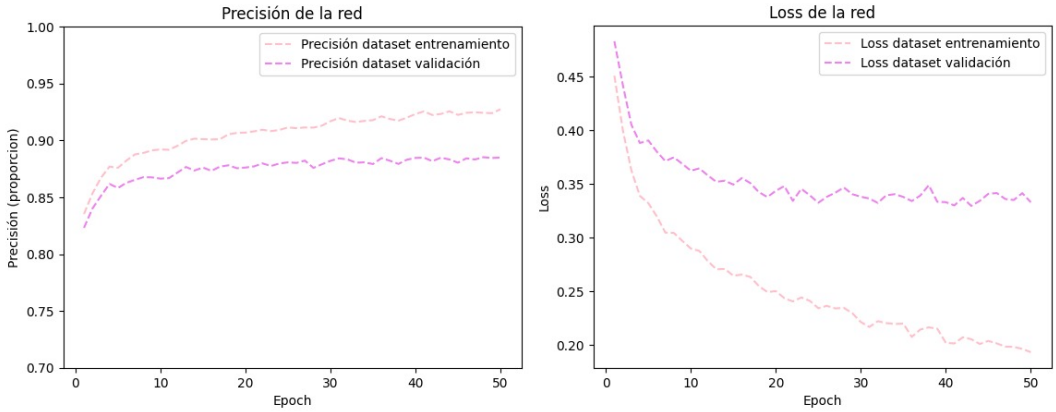
(a) Precisión del modelo con optimizador ADAM en entrenamiento y testeo

(b) Loss del modelo con optimizador ADAM en entrenamiento y testeo

Figure 2: Resultados del modelo con optimizador ADAM

Al observar los gráficos y los resultados obtenidos al cambiar el optimizador podemos notar la gran diferencia de épocas necesarias para alcanzar la misma precisión en comparativa al caso del modelo inicial. El optimizador ADAM alcanzo mas precisión en la época 15 que en las 50 épocas del modelo inicial utilizando el optimizador SGD. Podemos así afirmar que la elección de un buen optimizador es fundamental para reducir el coste y tiempo de entrenamiento de modelo, puedo así iterar mas rápido sobre mejoras del mismo.

C. Modelo con optimizador ADAM y dropout 0.35 (35%)



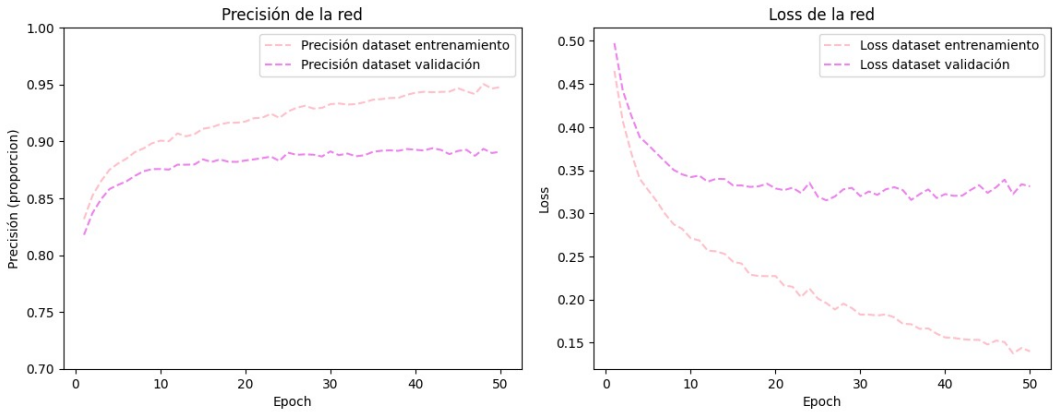
(a) Precisión del modelo con optimizador ADAM y dropout en entrenamiento y testeo

(b) Loss del modelo con optimizador ADAM y dropout en entrenamiento y testeo

Figure 3: Resultados del modelo con optimizador ADAM y dropout de **0.35**

Al analizar estos gráficos, podemos ver que al aumentar la tasa de dropout a 0.35, no se observa una mejora notable en el rendimiento del modelo. Este comportamiento es consistente con la idea de que, en este caso, la regularización adicional introducida por el dropout no es necesaria. Esta observación nos dice que probablemente no esta sobreajustando a los datos lo cual es positivo ya que indica que la red ya tiene una capacidad adecuada para generalizar sin la necesidad de regularización adicional.

D. Modelo con optimizador ADAM y lotes de 400



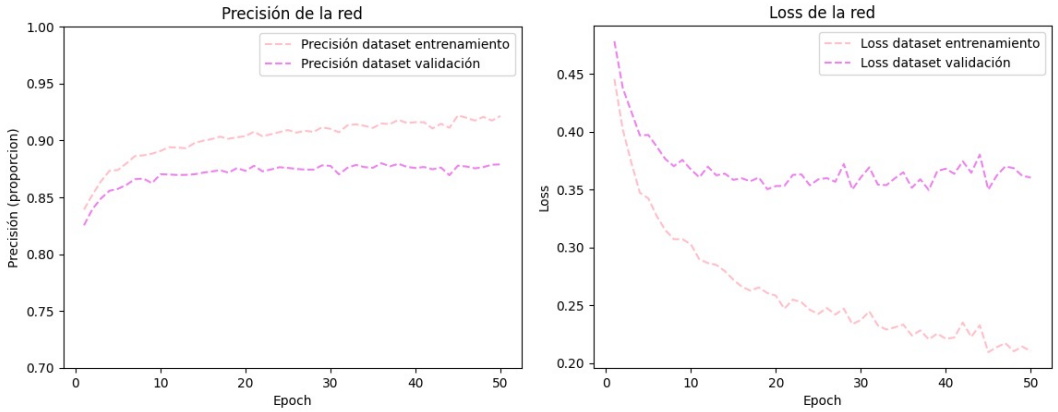
(a) Precisión del modelo con optimizador ADAM y lotes de 400 en entrenamiento y testeo

(b) Loss del modelo con optimizador ADAM y lotes de 400 en entrenamiento y testeo

Figure 4: Resultados del modelo con optimizador ADAM y lotes de 400

Estos gráficos reflejan que el modelo tiene una buena capacidad de generalización, sin diferencias significativas entre la precisión y la pérdida en los conjuntos de entrenamiento y validación, en este caso, aumentar el tamaño de los lotes no causo ningún efecto llamativo sobre el modelo.

E. Modelo con optimizador ADAM, 64 neuronas en la primera capa y 32 neuronas en la segunda capa



(a) Precisión del modelo con optimizador ADAM y 64-32 neuronas en entrenamiento y testeo

(b) Loss del modelo con optimizador ADAM y 64-32 neuronas en entrenamiento y testeo

Figure 5: Resultados del modelo con optimizador ADAM, 64 neuronas en la primera capa y 32 neuronas en la segunda capa

Al reducir el número de neuronas en las capas ocultas, se observa que esto no afecta significativamente la precisión del modelo. Esto nos permite ver que la cantidad de parámetros es la adecuada, lo que hace que el modelo sea más eficiente. El hecho de que con la mitad de parámetros podamos obtener la misma o muy similar precisión nos da una idea de un "sweet spot" en que minimizamos la cantidad de parámetros sin perder performance. Es decir, podemos alcanzar el mismo desempeño con un modelo mas sencillo, algo que siempre es útil.

IV. CONCLUSIÓN

A lo largo de este informe, se exploraron diferentes configuraciones de la red para analizar cómo los hiperparámetros afectan el rendimiento del modelo. Los resultados mostraron que el modelo es capaz de generalizar bien a nuevos datos, sin sobreajustarse, incluso al modificar ciertos parámetros clave. La principal diferencia fue la elección del optimizador ADAM, si bien esto no modifico fuertemente la precisión del modelo si logro poder reducir la cantidad de épocas de entrenamiento necesarias para obtener la máxima precisión, ahorrando computo y tiempo. A su vez, se logro reducir de cantidad de parámetros manteniendo el mismo performance, lo cual permite que el modelo sea mucho mas pequeño e igual de preciso.

En general, los resultados obtenidos destacan la efectividad de las redes neuronales multicapa para las tareas de clasificación.

* victorialopez23@mi.unc.edu.ar

[1] *Catedra de Redes Neuronales, Guia 12 - Clasificando Fashion-MNIST con PyTorch.*